Assignment 9:

```
import React, { useState, useEffect } from 'react';
import Header from './components/Header';
import FeedStats from './components/FeedStats';
import PostComposer from './components/PostComposer';
import CategoryFilter from './components/CategoryFilter';
import PostFeed from './components/PostFeed';
import Footer from './components/Footer';
import { initialPosts, AVAILABLE_CATEGORIES } from './data/mockPosts';

const LOCAL_STORAGE_KEY = 'kbtcoe_connect_posts_v1';

/**
 * App Component (Assignment 09 Root)
 * Demonstrates:
 * 1. useState for mutable community posts and filter states
 * 2. useEffect for persistent localStorage synchronization
 * 3. Immutable state updates for creating, liking, and deleting posts
 * 4. Derived live statistics and category filtering
 * 5. Reusable component composition
 */
export default function App() {
  // 1. Posts state with lazy initializer restoring from localStorage
  const [posts, setPosts] = useState(() => {
    try {
      if (typeof window !== 'undefined' && window.localStorage) {
        const savedJson = window.localStorage.getItem(LOCAL_STORAGE_KEY);
        if (savedJson) {
          const parsed = JSON.parse(savedJson);
          if (Array.isArray(parsed) && parsed.length > 0) {
            return parsed;
          }
        }
      }
    } catch (err) {
      console.warn('Failed to parse localStorage data, falling back to seed
posts:', err);
    }
    return initialPosts;
  });

  // 2. Active category filter state
  const [selectedCategory, setSelectedCategory] = useState('All');

  // 3. Persistent synchronization via useEffect
  useEffect(() => {
    try {
      if (typeof window !== 'undefined' && window.localStorage) {
        window.localStorage.setItem(LOCAL_STORAGE_KEY, JSON.stringify(posts));
      }
    } catch (err) {
      console.warn('LocalStorage synchronization failed:', err);
```

```
    }
  }, [posts]);

  // --- Handlers for Immutable State Operations ---

  // Add new post (prepends to top of feed immutably)
  const handleAddPost = (newPost) => {
    setPosts((prevPosts) => [newPost, ...prevPosts]);
  };

  // Toggle like/unlike on a specific post
  const handleToggleLike = (postId) => {
    setPosts((prevPosts) =>
      prevPosts.map((post) => {
        if (post.id === postId) {
          const nextLiked = !post.isLiked;
          return {
            ...post,
            isLiked: nextLiked,
            likes: nextLiked ? post.likes + 1 : Math.max(0, post.likes - 1)
          };
        }
        return post;
      })
    );
  };

  // Delete a user-created post
  const handleDeletePost = (postId) => {
    setPosts((prevPosts) => prevPosts.filter((post) => post.id !== postId));
  };

  // Reset to default sample posts
  const handleResetSeed = () => {
    if (window.confirm('Reset all posts to original demonstration data? Custom
posts will be cleared.')) {
      setPosts(initialPosts);
      setSelectedCategory('All');
      try {
        window.localStorage.removeItem(LOCAL_STORAGE_KEY);
      } catch (err) {
        console.warn('Failed to clear localStorage:', err);
      }
    }
  };

  // --- Derived Calculations (Reactive Statistics) ---
  const visiblePosts = selectedCategory === 'All'
    ? posts
    : posts.filter((post) => post.category === selectedCategory);

  const totalLikes = posts.reduce((sum, post) => sum + (post.likes || 0), 0);
  const userPostsCount = posts.filter((post) => post.isUserCreated).length;
```

```
  // Category counts map for badges
  const categoryCounts = AVAILABLE_CATEGORIES.reduce((acc, cat) => {
    if (cat === 'All') {
      acc[cat] = posts.length;
    } else {
      acc[cat] = posts.filter((p) => p.category === cat).length;
    }
    return acc;
  }, {});

  return (
    <div className="feed-app-shell">
      {/* Global Branding Header */}
      <Header />

      {/* Main Interactive Community Container */}
      <main className="container feed-main-content">
        {/* Page Hero Intro */}
        <section className="feed-hero-intro">
          <h1 className="feed-page-title">Campus Community Post Feed</h1>
          <p className="feed-page-subtitle">
            An interactive React community board demonstrating stateful post
updates,
            instant like/unlike toggles, category filtering, controlled post
composition, and local storage synchronization.
          </p>
        </section>

        {/* Live Aggregated Statistics */}
        <FeedStats
          totalPosts={posts.length}
          visibleCount={visiblePosts.length}
          totalLikes={totalLikes}
          userPostsCount={userPostsCount}
          activeCategory={selectedCategory}
          onResetSeed={handleResetSeed}
        />

        {/* Post Creation Controlled Form */}
        <PostComposer onAddPost={handleAddPost} />

        {/* Category Filter Tab Bar */}
        <section className="feed-filter-section" aria-label="Filter Posts by
Category">
          <div className="filter-header-row">
            <h3 className="filter-title">Explore Topics</h3>
            <span className="showing-indicator">
              Showing <strong>{visiblePosts.length}</strong> of {posts.length}
posts
            </span>
          </div>
          <CategoryFilter
            categories={AVAILABLE_CATEGORIES}
            selectedCategory={selectedCategory}
```

```
            onSelectCategory={setSelectedCategory}
            categoryCounts={categoryCounts}
          />
        </section>

        {/* Dynamic Post Feed Stream */}
        <PostFeed
          posts={visiblePosts}
          onToggleLike={handleToggleLike}
          onDeletePost={handleDeletePost}
          selectedCategory={selectedCategory}
          onResetFilter={() => setSelectedCategory('All')}
        />
      </main>

      {/* Global Footer */}
      <Footer />
    </div>
  );
}
```